



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Error Handling

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Error Handling

- Errors are caused by problems with the syntax or structure of the program.
- They typically occur before the program is even run, and they prevent the program from being executed at all.
- Errors in Python generally denote more serious problems that typically cannot be handled gracefully during runtime.
 - These can often lead to the termination of the program.



PROGRAMMING WITH PYTHON

Error Handling

- Error in Python can be of two types
 - Syntax errors
 - Logical Errors
- Errors are problems in a program due to which the program will stop the execution.



PROGRAMMING WITH PYTHON

Syntax Errors

- A syntax error occurs in Python when the interpreter is unable to parse the code due to the code violating Python language rules such as
 - inappropriate indentation
 - erroneous keyword usage
 - incorrect operator use.



PROGRAMMING WITH PYTHON

Syntax Errors

- Syntax errors prohibit the code from running, and the interpreter displays an error message that specifies the problem and where it occurred in the code.

```
x = 10  
if x == 10  
print("x is 10")
```

Cell In[1], line 2

```
if x == 10
```

^

SyntaxError: expected ':'



PROGRAMMING WITH PYTHON

Runtime Errors

- In Python, a runtime error occurs when the program is executing and encounters an unexpected condition that prevents it from continuing.



PROGRAMMING WITH PYTHON

Runtime Errors

- Runtime errors are also known as exceptions and can occur for various reasons such as
 - division by zero
 - attempting to access an index that is out of range
 - calling a function that does not exist.



PROGRAMMING WITH PYTHON

Logical Errors

- A logical error occurs in Python when the code runs without any syntax or runtime errors but produces incorrect results due to flawed logic in the code.



PROGRAMMING WITH PYTHON

Logical Errors

- These types of errors are often caused by
 - incorrect assumptions
 - an incomplete understanding of the problem
 - or the incorrect use of algorithms or formulas.



PROGRAMMING WITH PYTHON

Logical Errors

- Unlike syntax or runtime errors, logical errors can be challenging to detect and fix because the code runs without producing any error messages.
- The results may seem correct, but the code might produce incorrect output in certain situations.



PROGRAMMING WITH PYTHON

Logical Errors

- ```
def calculate_factorial(n):
 result = 1
 for i in range(1, n):
 result = result * i
 return result
```

```
print(calculate_factorial(5))
```

24



# PROGRAMMING WITH PYTHON

## Logical Errors

- ```
def calculate_factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result = result * i  
    return result
```

```
print(calculate_factorial(5))
```

120



PROGRAMMING WITH PYTHON

Errors handled in Python

```
import builtins
l=[name for name in dir(builtins)
    if isinstance(getattr(builtins, name), type)
    and
    issubclass(getattr(builtins, name), BaseException)]
print('List of Errors in Python')
for i in l:
    print(i)
```



PROGRAMMING WITH PYTHON

Runtime Errors

List of Errors in Python

ArithmeticError

AssertionError

AttributeError

BaseException

BaseExceptionGroup

BlockingIOError

BrokenPipeError

BufferError

BytesWarning

ChildProcessError

ConnectionAbortedError

ConnectionError

ConnectionRefusedError

ConnectionResetError

DeprecationWarning

EOFError

EncodingWarning

EnvironmentError

Exception

ExceptionGroup

FileExistsError

FileNotFoundError

FloatingPointError

FutureWarning

GeneratorExit

IOError

ImportError

ImportWarning

IndentationError

IndexError

InterruptedError

IsADirectoryError

KeyError

KeyboardInterrupt

LookupError

MemoryError

ModuleNotFoundError

NameError

NotADirectoryError

NotImplementedError

OSError

OverflowError

PendingDeprecationWarning

PermissionError

ProcessLookupError

RecursionError

ReferenceError

ResourceWarning

RuntimeError

RuntimeWarning

StopAsyncIteration

StopIteration

SyntaxError

SyntaxWarning

SystemError

SystemExit

TabError

TimeoutError

TypeError

UnboundLocalError

UnicodeDecodeError

UnicodeEncodeError

UnicodeError

UnicodeTranslateError

UnicodeWarning

UserWarning

ValueError

Warning

WindowsError

ZeroDivisionError



PROGRAMMING WITH PYTHON

Name Error

- A NameError in Python is raised when the interpreter encounters a variable or function name that it cannot find in the current scope.
- This can happen for a variety of reasons
 - misspelling a variable or function name
 - using a variable or function before it is defined
 - referencing a variable or function that is outside the current scope.



PROGRAMMING WITH PYTHON

Name Error

```
def calculate_sum(a, b):  
    total = a + b  
    return total
```

```
x = 5  
y = 10  
z = calculate_sum(x, w)  
print(z)
```

NameError

Traceback (most recent call last)

Cell In[7], line 3

```
1 x = 5  
2 y = 10  
----> 3 z = calculate_sum(x, w)  
4 print(z)
```

NameError: name 'w' is not defined



PROGRAMMING WITH PYTHON

Type Error

- The error is raised when an operation or function is applied to an object of an inappropriate type.
 - Performing arithmetic or logical operations on incompatible data types
 - Passing arguments of the wrong type to a function.



PROGRAMMING WITH PYTHON

Type Error

```
x = "10"  
y = 5  
Z = x + y  
print(z)
```

TypeError

Traceback (most recent call last)

Cell In[12], line 3

```
1 x = "10"  
2 y = 5  
----> 3 Z = x + y  
4 print(z)
```

TypeError: can only concatenate str (not "int") to str



PROGRAMMING WITH PYTHON

Index Error

- The error is raised in Python when an index of a sequence (such as a string, list, or tuple) that is out of range is tried to be accessed.
 - Trying to access an element that doesn't exist in the sequence
 - Trying to access an element at an index that is greater than or equal to the length of the sequence.



PROGRAMMING WITH PYTHON

Index Error

- ```
my_list = [100, 200, 300, 400, 500]
print(my_list[5])
```

```

IndexError Traceback (most recent call last)
Cell In[13], line 2
 1 my_list = [100, 200, 300, 400, 500]
----> 2 print(my_list[5])

IndexError: list index out of range
```



# PROGRAMMING WITH PYTHON

## Attribute Error

- The error is raised when an attribute or method of an object is accessed that does not exist or is not defined for that object.
- Misspelling the name of an attribute or method
- Trying to access an attribute or method that is not defined for the type of object that is being worked with.



# PROGRAMMING WITH PYTHON

## Attribute Error

- ```
my_string = "Hello, world!"  
my_string.reverse()
```

- ```

AttributeError Traceback (most recent call last)
Cell In[15], line 2
 1 my_string = "Hello, world!"
----> 2 my_string.reverse()

AttributeError: 'str' object has no attribute 'reverse'
```



# PROGRAMMING WITH PYTHON

## Floating Point Error

---

- It is a type of numerical error that occurs when representing real numbers in a computer.
- This is because computers can only store a finite number of digits, so they must approximate real numbers using a fixed number of bits.
- This approximation can introduce errors, which can accumulate over time.



# PROGRAMMING WITH PYTHON

## Floating Point Error

- `##Floating Point Error`

```
1.2 - 1.0
```

```
0.199999999999999996
```



# PROGRAMMING WITH PYTHON

## Key Error

- It is an error when a key is not found in a dictionary.

```
##Key Error
```

```
my_dictionary = {"name": "John", "age": 30, "city": "New York"}
print(my_dictionary["address"])
```

```

KeyError Traceback (most recent call last)
Cell In[24], line 2
 1 my_dictionary = {"name": "John", "age": 30, "city": "New York"}
----> 2 print(my_dictionary["address"])

KeyError: 'address'
```



- ## ##Overflow Error

```
f=calculate_factorial(1000)
f
```

Lekha A, CA, PESU



# PROGRAMMING WITH PYTHON

## Value Error

- The error is raised when a function gets an argument of correct type but improper value.

```
##Value Error
```

```
value = int("hello")
```

```

ValueError Traceback (most recent call last)
Cell In[30], line 1
----> 1 value = int("hello")

ValueError: invalid literal for int() with base 10: 'hello'
```



# PROGRAMMING WITH PYTHON

## Keyboard Interrupt Error

- The error is raised when the user does not complete the input method.

```
name=input('enter your name')
enter your name
Traceback (most recent call last):
 File "<pyshell#7>", line 1, in <module>
 name=input('enter your name')
KeyboardInterrupt
```



# PROGRAMMING WITH PYTHON

## Unbound Local Error

- The UnboundLocalError occurs when a local variable is referenced before it has been assigned a value within a function or method.



# PROGRAMMING WITH PYTHON

## Unbound Local Error

```
def factorial(n):
 for i in range(1, n):
 result*=i
 return result
```

```
factorial(5)
```

-----  
**UnboundLocalError** Traceback (most recent call last)

Cell In[5], line 1

----> 1 factorial(5)

Cell In[4], line 3, in factorial(n)

```
1 def factorial(n):
2 for i in range(1, n):
----> 3 result*=i
4 return result
```

**UnboundLocalError:** cannot access local variable 'result' where it is not associated with a value



# PROGRAMMING WITH PYTHON

## Recap

---

- Error Handling
- Types of errors
- Syntax Errors
- Logical Errors
- Runtime Errors
- Errors handled in Python



**THANK YOU**

---

**Lekha A**

Computer Applications